

## TP8 – AUTOMATES FINIS

On considère des mots sur un alphabet  $\Sigma$ . On pourra représenter dans ce sujet les lettres par le type `char` et les mots par des chaînes de caractères de type `char *`, terminées par le caractère nul.

Un automate est un modèle de machine qui permet de reconnaître linéairement en la taille d'un mot s'il correspond à un motif fixé (spoiler : une expression régulière).

### 1. REPRÉSENTATION D'UNE MACHINE À ÉTATS

Un automate est une machine qui à chaque instant se trouve dans un *état* donné. On note  $Q$  l'ensemble de tous les états. Dans ce sujet, on considère qu'il s'agit des entiers dans  $\llbracket 0, |Q| - 1 \rrbracket$ .

On représente en C un automate fini par une valeur du type `automaton`, donnée dans le fichier `automata.h`.

Le champ `first_transition` est un tableau qui pour chaque lettre  $a$  indique la position, dans `transitions` de la première transition étiquetée par  $a$ . Ainsi, si  $a$  est une lettre, les transitions étiquetées par  $a$  se trouvent entre les cases d'indices `first_transition[a]` et `first_transition[a+1] - 1`. La toute dernière case du tableau `first_transition` permet de conserver cette règle y compris pour la dernière lettre de l'alphabet.

**Question 1.** Que se passe-t-il quand aucune transition n'est étiquetée par une lettre ?

Certains états sont dits *acceptants*. Cette information est indiquée dans le champ `is_final`.

Initialement, quand il n'a encore lu aucune lettre, l'automate est dans l'état 0.

### 2. AUTOMATE DÉTERMINISTE

On suppose dans cette partie que les automates sont déterministes.

**Question 2.** Écrire une fonction `automaton_accepts` qui renvoie un booléen indiquant si, en partant de l'état 0 et en lisant un mot, un automate termine son calcul dans un état acceptant. Si l'automate est bloqué en cours de calcul (car il n'existe aucune transition portant la bonne étiquette), la fonction renvoie `false`.

(Indication : écrire d'abord comment, étant donné un état, une lettre et un automate calculer la valeur de l'état suivant s'il existe.)

#### Example 1. Le totomate

**Question 3.** Décrire un automate déterministe tel que les mots acceptés sont exactement ceux qui contiennent le facteur `toto`. On pourra remarquer qu'un automate se dessine naturellement comme un graphe étiqueté dont les nœuds sont les états et les arêtes sont les transitions.

**Question 4.** Décrire un automate qui accepte exactement les mots de  $L_0 = \{a^n b^m \mid n, m \in \mathbb{N}\}$ .

**Question 5.** Décrire un automate qui accepte exactement les mots de  $L_0^*$ .

**Question 6.** Qui n'a pas programmé en C les tests avec ces automates ?

### 3. LECTURE D'UN AUTOMATE

Au lieu d'écrire un automate directement en C, j'aimerais écrire mes automates sous la forme d'un fichier texte :

```

finals: 4;
0 ->[t] 1;
1 ->[o] 2;
1 ->[t] 1;
2 ->[t] 3;
3 ->[t] 1;
3 ->[o] 4;
4 -> 4;
0 ->![t] 0;
1 ->![to] 0;
2 ->![t] 0;
3 ->![to] 0;

```

La première ligne indique la liste des états acceptants, séparés par des espaces. Chaque ligne suivante est en général de la forme  $q_1 \rightarrow[\text{lettre}] q_2$ ; pour une transition  $q_1 \rightarrow^a q_2$ . La lettre peut ne pas être indiquée, on a alors une ligne  $q_1 \rightarrow q_2$ ; qui indique que l'on crée *toutes* les transitions  $q_1 \rightarrow^a q_2$ , lorsque  $a$  décrit l'alphabet. Enfin, on peut également écrire des lignes de la forme  $q_1 \rightarrow![\text{lettres}] q_2$ ; pour indiquer que l'on crée les transitions de  $q_1$  vers  $q_2$  pour toutes les lettres sauf celles indiquées dans les crochets.

Le programme principal s'utiliserait alors ainsi :

```
./automaton fichier_automate
```

SHELL

Ce programme lit sur l'entrée standard des mots, un par ligne, et pour chacun d'entre eux indique s'il est reconnu par l'automate.

**Question 7.** Écrire la fonction qui, à partir d'un tel fichier, crée la structure de type `automaton` correspondante. (C'est une fonction un brin technique.)

## 4. LANGAGES LOCAUX

**Question 8.** Soit  $L$  un langage local. Construire un automate qui reconnaît exactement les mots de  $L$ . Votre automate devrait avoir  $|\Sigma| + 1$  états et vérifier le fait que toutes les transitions étiquetées par une même lettre partent d'un même état.

**Question 9.** Réciproquement, montrer qu'un automate tel que toutes les transitions étiquetées par une même lettre partent d'un même état reconnaît exactement les mots d'un langage local, à définir.

## 5. AUTOMATES NON DÉTERMINISTES

Désormais, on considère des automates qui peuvent être non déterministes. L'automate accepte alors un mot *s'il existe au moins un chemin* étiqueté par les lettres du mot, allant de l'état initial jusqu'à un état final. S'il existe d'autres chemins (non acceptants ou bloqués), ces autres chemins ne sont pas pris en compte.

**Question 10.** Écrire une fonction `automaton_nd_accepts` qui explore tous les chemins et indique si le mot est accepté. Une fonction intermédiaire utile pourrait être la fonction `automaton_nd_accepts_from` qui indique si un mot est accepté mais en partant d'un état quelconque fixé, pas nécessairement 0.

Une deuxième approche pour explorer tous les chemins consiste à garder en mémoire, à chaque instant, la liste des états dans lesquels l'automate se trouve. Ceci revient à faire avancer en parallèle tous les chemins en cours d'exploration. Une telle liste peut se représenter par un tableau qui à chaque état associe un booléen.

**Question 11.** Écrire une fonction `automaton_nd_accepts_parallel` qui utilise cette idée.